

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
ΜΑΘΗΜΑ: ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΣΤΟ
ΔΙΑΔΙΚΤΥΟ & ΣΤΟΝ ΠΑΓΚΟΣΜΙΟ ΙΣΤΟ
ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ: 2024-2025
ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ ΕΞΑΜΗΝΟΥ**

**Θέμα: Ανάπτυξη Ολοκληρωμένης 3-tier
Διαδικτυακής Εφαρμογής Διαχείρισης Ιατρικών
Ραντεβού
Ονοματεπώνυμο: Αντώνης Μπαρδάνης
Αριθμός Μητρώου: Π21110
Ημερομηνία: 22/11/2025**

1. Εισαγωγή & Στόχοι

Σκοπός της τελικής εργασίας ήταν η επέκταση και ολοκλήρωση της εφαρμογής διαχείρισης ιατρικών ραντεβού σε ένα πλήρες σύστημα αρχιτεκτονικής τριών επιπέδων (3-tier architecture). Η εφαρμογή υλοποιήθηκε με χρήση τεχνολογιών Java EE (Servlets, JSP), συνδέεται με σχεσιακή βάση δεδομένων (MySQL) και διαχωρίζει τη λογική της σε επίπεδα Μοντέλου, Ελέγχου και Προβολής (MVC).

Ιδιαίτερη έμφαση δόθηκε στην ασφάλεια (κρυπτογράφηση κωδικών, διαχείριση συνεδριών/sessions), στον διαχωρισμό ρόλων (Admin, Doctor, Patient) και στην υλοποίηση επιχειρησιακής λογικής (π.χ. κανόνες ακύρωσης ραντεβού).

2. Αρχιτεκτονική Συστήματος

Η εφαρμογή ακολουθεί το πρότυπο Model-View-Controller (MVC):

Model (Μοντέλο - Επίπεδο Δεδομένων):

Περιλαμβάνει τις κλάσεις User, Patient, Doctor, Appointment που αναπαριστούν τα δεδομένα.

Περιλαμβάνει τα DAO (Data Access Objects) (UserDAO, AppointmentDAO) που εκτελούν τα SQL ερωτήματα προς τη βάση δεδομένων.

Περιλαμβάνει την κλάση DBConnection για τη διαχείριση της σύνδεσης JDBC.

View (Προβολή - Επίπεδο Διεπαφής):

Υλοποιήθηκε με χρήση JSP (JavaServer Pages) και HTML/CSS.

Οι σελίδες είναι οργανωμένες σε φακέλους (admin/, doctor/, patient/) για καλύτερη δομή και ασφάλεια.

Η προβολή των δεδομένων γίνεται δυναμικά με χρήση scriptlets Java.

Controller (Ελεγκτής - Επίπεδο Επεξεργασίας):

Υλοποιήθηκε με Java Servlets.

Τα Servlets (AuthServlet, RegisterServlet, AdminServlet, ActionServlet) δέχονται τα αιτήματα HTTP (POST/GET), καλούν τις μεθόδους του Model και ανακατευθύνουν τον χρήστη στο κατάλληλο View.

3. Σχεδιασμός Βάσης Δεδομένων & Ασφάλεια

Η βάση δεδομένων medical_db επανασχεδιάστηκε για να υποστηρίξει τις νέες λειτουργίες.

Οι πίνακες που χρησιμοποιήθηκαν είναι:

users: Αποθηκεύει τα στοιχεία εισόδου. Ο κωδικός πρόσβασης (password) αποθηκεύεται κρυπτογραφημένος (SHA-256 Hash) και όχι ως απλό κείμενο, διασφαλίζοντας το απόρρητο των χρηστών.

patients / doctors: Πίνακες που συνδέονται με τον users (1-to-1 relationship) και κρατούν εξειδικευμένα στοιχεία (ΑΜΚΑ, Ειδικότητα).

appointments: Συνδέει Γιατρούς και Ασθενείς. Περιλαμβάνει πεδίο status (AVAILABLE, BOOKED, CANCELLED) για την παρακολούθηση της κατάστασης του ραντεβού.

Μηχανισμός Ασφαλείας: Δημιουργήθηκε η κλάση HashUtils, η οποία χρησιμοποιεί τον αλγόριθμο SHA-256 για να μετατρέπει τους κωδικούς σε hash strings πριν την αποθήκευση στη βάση ή τον έλεγχο κατά το Login.

4. Υλοποίηση Λειτουργιών ανά Ρόλο

4.1 Διαχειριστής (Admin)

Login: Είσοδος με ειδικά διαπιστευτήρια.

Διαχείριση Ιατρών: Μέσω του AdminServlet, ο διαχειριστής μπορεί να εγγράψει νέους ιατρούς στο σύστημα, ορίζοντας Username, Password (που κρυπτογραφείται αυτόματα) και Ειδικότητα.

Dashboard: Προβολή ειδικού πίνακα ελέγχου (admin/dashboard.jsp).

4.2 Ιατρός (Doctor)

Διαχείριση Διαθεσιμότητας: Ο ιατρός επιλέγει ημερομηνία και ώρα για να δημιουργήσει νέα διαθέσιμα slots (AVAILABLE) στη βάση δεδομένων.

Προβολή Προγράμματος: Μέσω της μεθόδου getDoctorSchedule του DAO, ο ιατρός βλέπει μια λίστα με τα ραντεβού του, το όνομα του ασθενή (αν έχει κλειστεί) και την κατάσταση.

Ακύρωση: Δυνατότητα ακύρωσης ραντεβού.

4.3 Ασθενής (Patient)

Εγγραφή (Registration): Υλοποιήθηκε το RegisterServlet που επιτρέπει σε νέους χρήστες να δημιουργήσουν λογαριασμό ασθενή. Το σύστημα ελέγχει αν το Username ή το ΑΜΚΑ υπάρχουν ήδη.

Κλείσιμο Ραντεβού: Ο ασθενής βλέπει λίστα με τα διαθέσιμα ραντεβού όλων των γιατρών (status='AVAILABLE') και μπορεί να κάνει κράτηση (ActionServlet -> book).

Ιστορικό & Ακύρωση: Προβολή προσωπικού ιστορικού. Ο ασθενής μπορεί να ακυρώσει ραντεβού μόνο εάν η ημερομηνία διεξαγωγής απέχει περισσότερο από 3 ημέρες από την τρέχουσα, όπως ορίζει η εκφώνηση (έλεγχος ενσωματωμένος στο AppointmentDAO).

5. Παραδείγματα Εκτέλεσης (Screenshots)

Ακολουθούν στιγμιότυπα από τη λειτουργία της ολοκληρωμένης εφαρμογής:

Εικόνα 1: Αρχική Σελίδα & Login

Medical System Login

Username:

Password:

Login

Εγγραφή Ασθενή

Εγγραφή Νέου Ασθενή

Username:

Password:

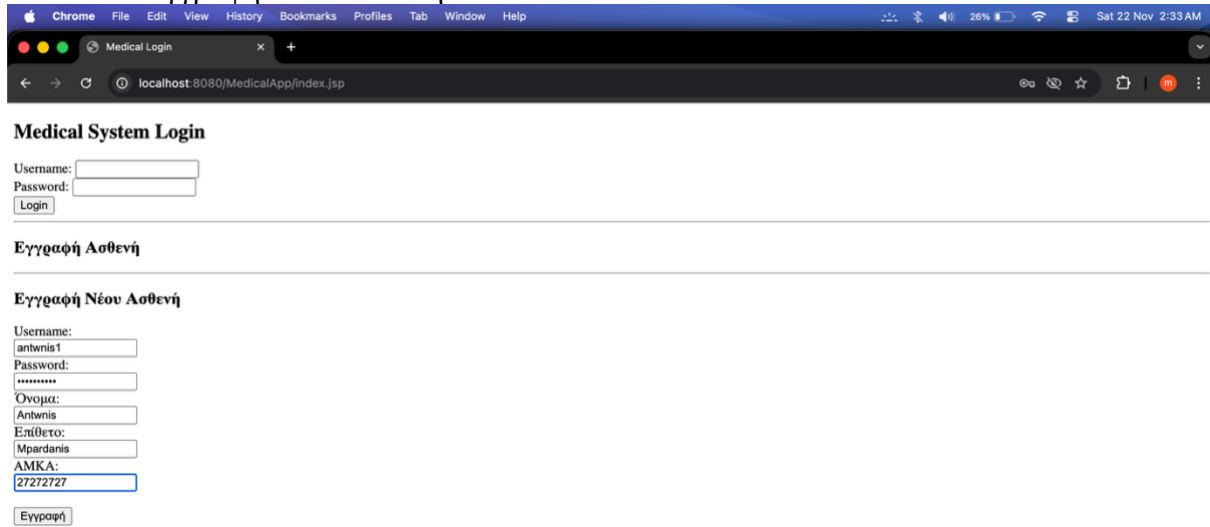
Όνομα:

Επίθετο:

ΑΜΚΑ:

Εγγραφή

Εικόνα 2: Εγγραφή Νέου Ασθενή



Medical System Login

Username:

Password:

Login

Εγγραφή Ασθενή

Εγγραφή Νέου Ασθενή

Username:

Password:

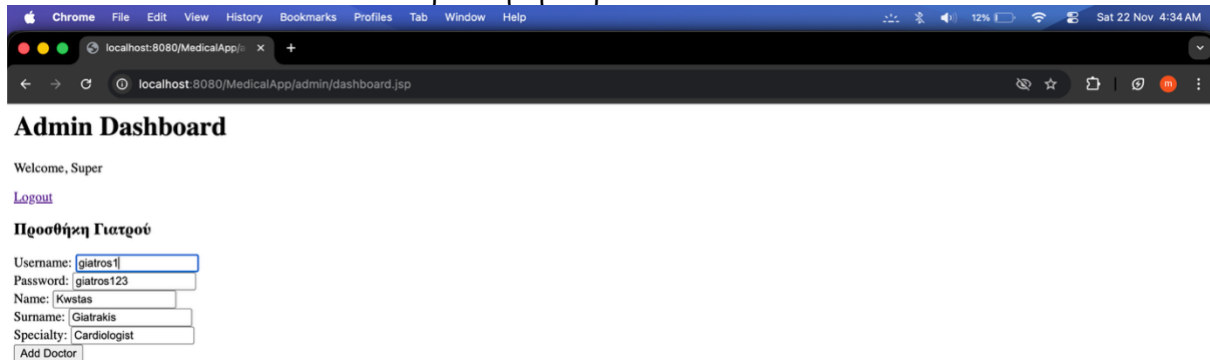
Όνομα:

Επίθετο:

AMKA:

Εγγραφή

Εικόνα 3: Admin Dashboard - Προσθήκη Ιατρού



Admin Dashboard

Welcome, Super

[Logout](#)

Προσθήκη Γιατρού

Username:

Password:

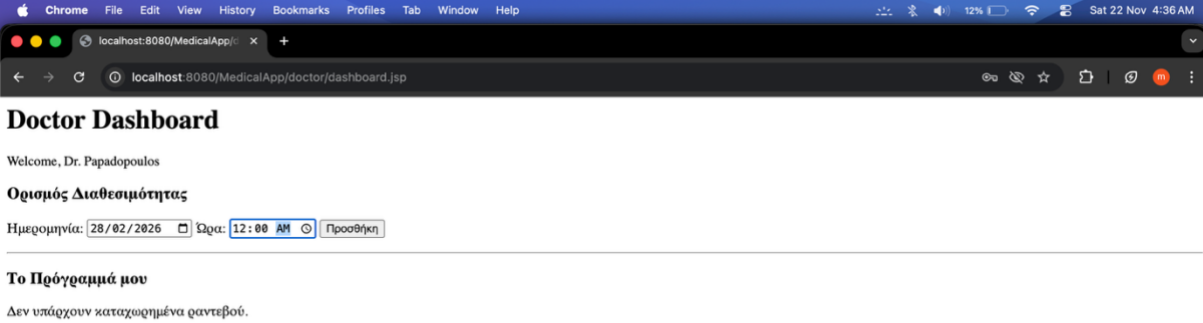
Name:

Surname:

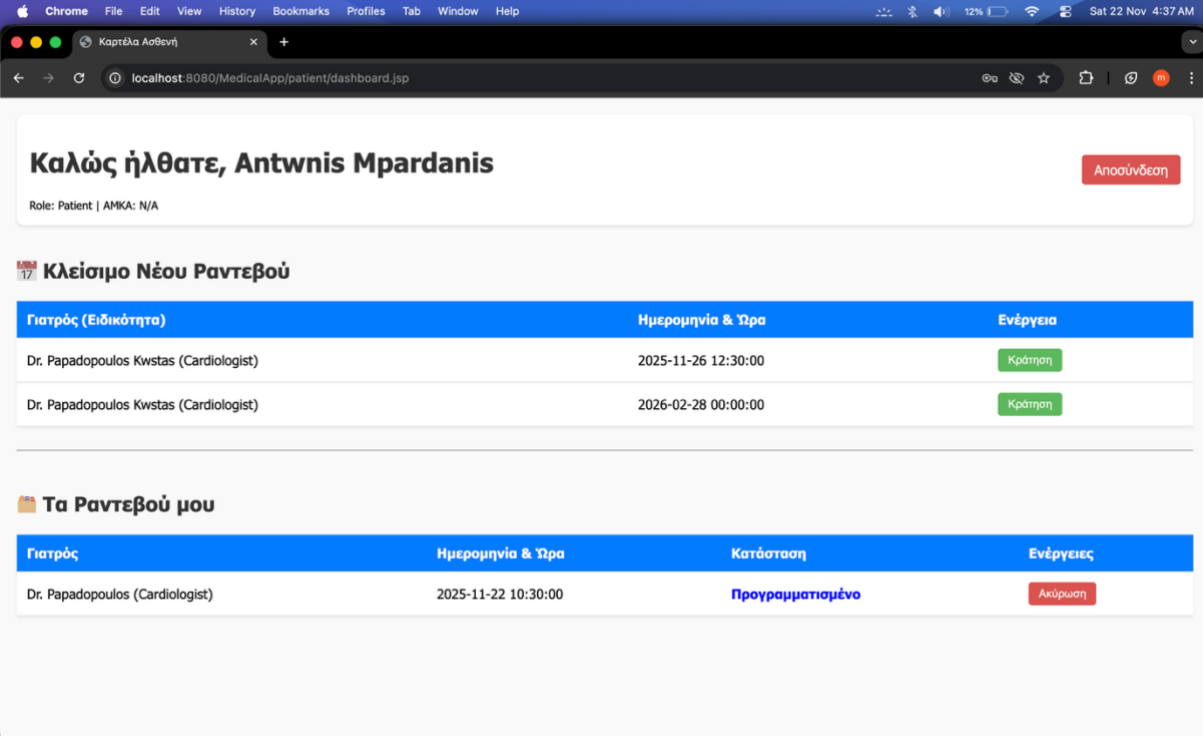
Specialty:

Add Doctor

Εικόνα 4: Doctor Dashboard - Προσθήκη Διαθεσιμότητας & Λίστα



Εικόνα 5: Patient Dashboard - Διαθέσιμα Ραντεβού & Ιστορικό



Εικόνα 6: Βάση Δεδομένων - Κρυπτογραφημένοι Κωδικοί
Εμφάνιση του πίνακα users όπου φαίνονται τα hashed passwords.

The screenshot displays the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'medical_db' database selected. The 'Query' editor in the center contains the SQL statement: `SELECT * FROM users`. The 'Result Grid' at the bottom shows the data returned by the query, which includes columns for `id`, `username`, `password`, `salt`, `name`, `surname`, and `role`. The data is as follows:

id	username	password	salt	name	surname	role
1	admin	243be5189bcd77244b6f04eb1da598744b7e...		Super	Admin	ADMIN
2	doc1	0ba2b742168a15c761ee1a5e25506795382c...		Doctoras	Doctordis	PATIENT
4	giatros1	44432afc977dc150ca2890b4aa5a474b64ba30...		Kwstas	Papadopoulos	DOCTOR
5	asthenis1	48e01b77ea2a8f24cd230a76b6f4c327ed3e2...		Antwnis	mpardanis	PATIENT
6	antwnis1	35a32d47aep1a966d82bd595652b14fbbd629...		Antwnis	mpardanis	PATIENT

The 'Action Output' pane at the bottom shows the sequence of database actions performed, including creating the database, using the database, creating tables, and inserting data into the 'users' table.

6. Συμπεράσματα

Η εργασία ολοκληρώθηκε επιτυχώς, καλύπτοντας όλες τις απαιτήσεις για μια σύγχρονη διαδικτυακή εφαρμογή. Η χρήση της αρχιτεκτονικής 3-tier επέτρεψε την εύκολη συντήρηση και επέκταση του κώδικα. Η ενσωμάτωση κρυπτογράφησης και ελέγχων συνεδρίας (sessions) διασφάλισε την ακεραιότητα και ασφάλεια του συστήματος, ενώ η χρήση Servlets και JDBC προσέφερε αποδοτική επικοινωνία με τη βάση δεδομένων.